



MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Algoritmos e Estrutura de Dados II

Anotações de Aula

Profa Melise Maria Veiga de Paula

Lista

Lista é uma estrutura que permite representar um conjunto de dados de forma a preservar a relação de ordem linear que existe entre estes dados. Uma lista linear é composta de nós, os quais podem conter, cada um deles, um dado primitivo ou um dado composto, ou seja, um nó pode conter um inteiro, uma string, um elemento de um tipo abstrato de dados ou qualquer outro elemento. Formalmente, uma lista pode ser definida como:

Conjunto de n nós $x_1, x_2, \dots, x_n$, (com $n \geq 0$) organizados estruturalmente de forma a refletir as posições relativas dos mesmos. Se $n > 0$ então x_1 é o primeiro nó; para $1 < k < n$, o nó k é precedido pelo nó x_{k-1} e é seguido do nó x_{k+1} ; e x_n é o último nó. Quando $n = 0$, dizemos que a lista é vazia. Deste modo, em uma lista, a relação de precedência entre os nós deve ser preservada.

Exemplos de listas lineares:

- lista dos dias da semana
- lista de funcionários cadastrados em uma empresa em ordem alfabética do nome
- lista do número de matrícula e nome de alunos ordenada pela nota do aluno

Para que uma lista possa ser manipulada, é essencial definir algumas operações, por exemplo:

1. Inicializar (criar) uma lista
2. Inserir um nó na lista (em qualquer posição da lista)
3. Remover um nó da lista (em qualquer posição da lista)
4. Consultar um nó da lista

Como acontece em algumas estruturas de dados, a estrutura “lista linear” e as operações sobre a lista podem ser implementadas de diversas maneiras. Vamos discutir algumas delas a seguir:

Primeira estratégia: implementação por contigüidade física

Nesta estratégia, a sequência da memória do computador é usada para representar a ordem dos nós da lista. Endereços físicos adjacentes na memória representam nós logicamente adjacentes na lista. Os vetores podem ser considerados a forma mais usual de implementar uma lista por contigüidade física. Por este motivo, esta estratégia costuma ser denominada lista estática sequencial. A seguir, um exemplo de uma lista linear estática sequencial implementada com um vetor.

Considere L como sendo uma lista contendo os números de matrículas dos alunos em uma disciplina:

Lista L

M1	M2	M3	M4	M5
----	----	----	----	----



MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Na lista, M1 é o primeiro nó e ocupa a posição 0 do vetor. Note que a posição do vetor representa também a ordem do nó na lista L. Vamos discutir agora, as implicações quando trabalhamos com listas usando esta estrutura estática seqüencial:

Imagine que você queira remover o segundo nó da lista, suponha que o aluno com matrícula M2 não faz esta disciplina. Neste caso, ao remover o nó M2, todos os demais devem ser deslocados para que M3 passe a ser o novo sucessor de M1. Em uma lista de tamanho N, uma quantidade aproximada de N operações deverão ser realizadas. A mesma coisa acontece quando temos que inserir um nó no começo ou no meio da lista. Por outro lado, o acesso aos nós é direto, ou seja, se você conhece a posição do nó na lista, basta usar o índice correspondente do elemento no vetor para recuperá-lo.

Deste modo, é possível perceber que esta estratégia, embora facilite o acesso aos nós, dificulta a inclusão e remoção de nós. Além disso, como usamos um vetor, é necessário definir a priori o tamanho máximo da lista. Vamos para outra estratégia.

Segunda estratégia: implementação estática com encadeamento

Nesta estratégia, os nós da lista continuam sendo armazenados em um vetor. Contudo, a regra de precedência desses nós, ou seja, a ordem dos nós na lista não é definida a partir dos endereços físicos e sim a partir de uma informação adicional. Mas por que chamar Lista encadeada? A palavra encadeada significa que a ordem dos elementos na lista é feita por encadeamento.

Deste modo, para usar esta estratégia, todo nó deve ter, além da informação (dado) propriamente dita, uma informação adicional que determina a posição do nó na lista. Neste caso, a ordem dos elementos no vetor pode ser diferente da ordem dos nós na lista.

Uma das estratégias é manter a posição do elemento na lista. Veja o exemplo da lista L2, na figura, os índices em vermelho representam os índices dos elementos no vetor, já os números em azul representam a posição dos nós na lista.

Neste caso, o primeiro elemento da lista L2 não é M1, mas o M3. A ordem dos elementos na lista L2 é M3->M1->M4->M2->M5. Note que esta ordem é, de fato, diferente da ordem dos elementos no vetor.

Lista L2

0 M1 2	1 M2 4	2 M3 1	3 M4 3	4 M5 5
----------------------------	----------------------------	----------------------------	----------------------------	----------------------------



MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Outra estratégia é fazer o encadeamento determinando o índice do próximo elemento na lista. Além do próximo elemento, também pode-se manter o índice do antecessor. Veja L2 novamente. Agora, aparecem duas informações: o índice do sucessor (amarelo) e do antecessor do elemento no vetor (vermelho). Na L2, o sucessor de M1 é o nó que está na posição 3 do vetor, o M4. Já o sucessor de M3 é M1, pois é ele que está na posição 0 do vetor.

Lista L2

0	1	2	3	4
M1	M2	M3	M4	M5
2	3	-1	0	1
3	4	0	1	-1

Mas como saber quais os elementos do vetor são o primeiro e o último nós respectivamente? O último nó não tem um sucessor, na maioria das implementações disponíveis, é usado o -1 para representar o sucessor do último nó (isto é uma convenção). A mesma estratégia é utilizada para representar o antecessor do primeiro nó.

A ordem dos elementos na L2 é:
M3->M1->M4->M2->M5

Mas diferentes estratégias podem ser encontradas. Uma delas é usar uma variável de nome “*com*” que vai guardar o índice do primeiro nó da lista. Na L2, por exemplo, *com* = 2.

Agora considere a mesma situação anterior, temos que retirar da lista o nó correspondente à matrícula M2. O nó contendo esta informação dever ser “procurado” a partir do primeiro nó da lista. Depois de removido o nó M2, a lista L2 ficaria assim:

Lista L2

M3->M1->M4->M5

com = 2

0		2	3	4
M1		M3	M4	M5
2		-1	0	3
3		0	4	-1

Observem que, neste caso, é importante definir uma estratégia para identificar quais elementos (posições) do vetor estão disponíveis para receber um novo nó da lista.

Exercício

1. Elabore um programa que use uma lista encadeada de nós a partir de um vetor. Cada nó deve conter, além da informação (um inteiro positivo), um campo que indica a posição do próximo elemento na lista. Considere que os elementos da lista estão ordenados. Análise o problema, crie um TAD e decida qual a estrutura do novo TAD e quais operações serão necessárias. Implemente estas operações.



MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Terceira estratégia: implementação por alocação dinâmica de nós

A segunda estratégia resolve o problema relacionado ao desempenho das operações de inclusão e exclusão de nós da lista. Por outro lado, esta não consegue resolver o problema do tamanho da lista. Ainda temos que determinar, a priori, o tamanho máximo da lista. Desta forma, é necessário definir outra estratégia. Uma solução para este problema é usar a alocação dinâmica.

Na alocação dinâmica de memória, o programador não precisa definir, a priori, o tamanho da memória necessária para resolver o problema. A memória alocada não é estática, ou seja, a memória alocada pode variar durante a execução do programa. Por isso, nos referimos a este tipo de alocação como sendo “*Alocação Dinâmica*” (a alocação da memória é feita de forma dinâmica): à medida que a solução vai sendo executada, caso seja necessário, a memória é alocada.

Em C, a alocação dinâmica é feita através dos ponteiros. Um ponteiro é um endereço de memória, ou seja, um ponteiro guarda um endereço da memória que deverá conter efetivamente o valor a ser usado no problema. Veja o exemplo abaixo:

```
int v;
```

```
int *p;
```

- v é uma variável do tipo inteiro, ou seja, no local da memória identificado por v , será armazenado um valor inteiro, por exemplo, se $v = 10$, no endereço identificado por v , teremos o valor 10 armazenado.
- p é um ponteiro, ou seja, v armazena um endereço de memória que contém um valor inteiro, poderíamos fazer, por exemplo:
- $p = \&v$

Imagine que o endereço de v na memória seja 1245839, então, $p = 1245839$ e $*p = 10$, ou seja, o valor “apontado” por p é 10 (o valor contido no endereço armazenado em p). A figura abaixo ilustra uma representação da memória. Perceba que, tanto v quanto p são variáveis. Contudo, p e v são variáveis de tipos diferentes.

p	v
1245839	10

No comando $p = \&v$, o espaço de memória apontado por p já existe, pois v já foi alocada. Mas podemos usar o conceito de ponteiros com a alocação dinâmica. Neste caso, declaramos o ponteiro e alocamos dinamicamente a memória para receber os valores que serão apontados por p e manipulados no programa.

A biblioteca `stdlib.h` disponibiliza várias funções para alocação dinâmica, dentre elas, vamos conhecer neste momento, as funções `malloc` e `free`.

A função `malloc` aloca um bloco de bytes consecutivos na memória do computador e devolve um ponteiro para esse lugar. A função `free` libera a porção de memória alocada por `malloc`. Um exemplo simples;



MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

```
int *p;  
p = (int *) malloc (sizeof (int)); // p é o endereço de um espaço capaz de armazenar um  
inteiro  
if (!p) printf("\n Não tem memória");  
else {  
    *p = 150;  
    printf ("%d", *p);  
    free (p);  
}
```

A expressão `sizeof (int)` retorna a quantidade de bytes de um `int`. A função `malloc` devolve um ponteiro "genérico" (ou seja, do tipo `void`) para um bloco de bytes consecutivos. Como queremos um ponteiro para um inteiro, é preciso "moldar" o resultado. É este o papel do *molde de tipo* em C também chamado "*type cast*" (`int *`) que precede `malloc`.

Se a memória do computador estiver esgotada, `malloc` devolve `NULL`. Este teste deve ser realizado antes da utilização do ponteiro `p`.

O efeito de `free(p)` é avisar o sistema de que o bloco de bytes apontado por `p` está livre. A próxima chamada de `malloc` poderá tomar posse desses bytes.

A alocação da memória é ilustrada abaixo:

`int *p;` (declaração da variável tipo ponteiro para inteiro);

p	
---	--

O valor de `p` ainda não foi especificado, ou seja, `p` ainda não aponta para outro endereço na memória.

`p = (int *) malloc (sizeof (int));`

p	11111
11111	

O endereço 11111 é alocado na memória e atribuído a `p`. O endereço 11111 deve ter o tamanho suficiente para armazenar um inteiro, pois foi dito que `p` é um ponteiro para inteiro.

`*p = 150;`

p	11111
11111	150

O valor 150 é atribuído ao espaço de memória apontado por `p`.

`free (p);`

p	
---	--

O espaço de memória apontado por `p` é liberado, mas `p` continua existindo!!!

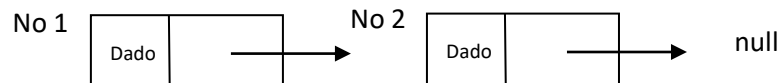


MINISTÉRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Mas como usar um ponteiro para manipular uma lista???

A idéia é que o ponteiro seja usado para definir o encadeamento dos nós da lista. Nesta estratégia, todo nó é composto por dois campos: o dado propriamente dito e um ponteiro que deve apontar para o endereço de memória do próximo nó da lista. Na figura abaixo, o nó 1 contém um ponteiro que aponta para o nó 2. Como nó 2 é o último da lista. O seu prox é definido como sendo NULL.



Em C, quando uma variável é composta, ou seja, quando uma variável possui uma estrutura interna, usamos struct. Logo, nossos nós serão criados a partir de um struct. Suponha que vamos resolver um problema usando uma lista de inteiros. Nossos nós terão a seguinte forma:

```
struct no {
    int dado;
    struct no *prox;
}
```

Observem que prox não é um ponteiro para um inteiro, prox é um ponteiro para outro nó, ou seja, outro struct no.

Como vamos usar este struct várias vezes, é interessante criar um tipo a partir dele. Vamos criar o tipo tno:

```
typedef struct no {
    int dado;
    struct no *prox;
} tno;
```

Mas como criar nossa lista a partir de tno? Temos que fazer algumas considerações:

- não conhecemos o tamanho da nossa lista;
- estamos supondo que os nós serão ordenados de forma crescente;

Como nossos nós possuem um ponteiro para o próximo nó, para alcançar um determinado nó da lista, basta seguir o encadeamento começando sempre do primeiro nó. Portanto, para manipular qualquer nó, basta que seja informado o primeiro nó da lista. A partir dele, podemos alcançar qualquer outro. Desta forma, nossa lista será representada pelo primeiro nó, ou melhor, nossa lista será representada por um ponteiro para o primeiro nó da lista.

Inicialmente, nossa lista terá apenas esta informação. Mas futuramente, podemos trabalhar com outros tipos de cenários e nossa lista pode conter outras informações. Portanto, prevendo futuras implementações, vamos criar nossa lista como sendo um struct.



MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

```
struct lista {  
    tno *com;  
}
```

Nos nossos problemas, podemos criar várias listas e a idéia é que o conceito de tipo abstrato de dados seja usado. Portanto, seguindo esta idéia, vamos criar um tipo baseado no struct.

```
typedef struct lista {  
    tno *com;  
} tlista;
```

Considerando as suposições acima, a seguir, uma possível implementação para as operações de inclusão de nó, exclusão de um nó e exibição do conteúdo dos nós na lista. Vale lembrar que, antes de qualquer utilização da lista, é necessário inicializá-la.

```
void iniciaLista(tlista *l) {  
    l->com = (tno *) NULL;  
}
```

No código acima, não é necessário usar o modificador de tipo (typecast) (tno *) , fizemos assim para garantir que l->com é realmente compatível com o tipo tno.

Nessa estratégia, vamos usar duas variáveis auxiliares, aux e ant. O novo nó sempre vai ser inserido antes de aux e depois de ant. Se o novo nó for o novo primeiro nó da lista, a variável ant será NULL, pois aux é o primeiro nó da lista. Vamos ter que considerar isso na implementação.

```
int insereLista (tlista *l, int valor){  
    tno *aux, *p, *ant;  
    p = (tno*)malloc(sizeof(tno));  
    if (!p) return 0;  
    else  
    {  
        p->dado = valor;  
        p->prox = (tno *)NULL;  
        aux = l->com;  
        ant = (tno *)NULL;  
        while ((aux) && (aux->dado < p->dado)) {  
            ant = aux;  
            aux = aux->prox;  
        }  
        if (!ant) {// novo nó é o novo primeiro nó da lista  
            p->prox = l->com;  
            l->com = p;  
        }  
        else {
```



MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

```
        ant->prox = p;
        p->prox = aux;
    }
    return 1;
}
}
```

```
int removelista(tlista *l, int dado){
    tno *no, *aux, *ant;
    aux = l->com;
    ant = NULL;
    if (listavazia(*l)) return 0; // lista vazia
    while (aux && aux->dado != dado){
        ant = aux;
        aux = aux->prox;
    }
    if (aux){ // achou o no
        if (ant){ // não é o primeiro
            ant->prox = aux->prox;
        }
        else {
            l->com = aux->prox;
        }
        free(aux);
        return 1;
    }
    return 0;
}
```

```
void mostraLista(tlista l){
    tno *aux;
    aux = l.com;
    if (!aux) printf ("\nLista vazia\n");
    else {
        while (aux) {
            printf("\n %d",aux->dado);
            aux = aux->prox;
        }
    }
}
```

Agora, vamos trabalhar com uma lista um pouco diferente, cada nó da lista terá um ponteiro para o próximo elemento e um ponteiro para o elemento anterior. Além disso, nossa lista será definida com um ponteiro para o começo e um ponteiro para o fim.



MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Esta lista é denominada duplamente encadeada pois permite o percurso tanto do começo para o fim, quanto do fim para o começo. Por isso, é comum incluir no TAD lista duplamente encadeada a operação para percorrer a lista do fim para o começo. Isso também justifica o fato de a lista ser definida com um ponteiro para o começo e para outro para o fim.

Portanto, teremos as seguintes especificações de TAD:

```
typedef struct no {  
    int dado;  
    struct no *prox;  
    struct no *ant;  
} tno;
```

```
typedef struct lista {  
    tno *com; tno *fim;  
} tlista;
```

No nosso caso, vamos trabalhar com uma lista ordenada de inteiros, logo, vamos chamar essas operações de percurso de: imprime lista crescente e imprime lista decrescente.

```
void crialista(tlista *l);  
int inserelista(tlista *l, int dado);  
int removelista(tlista *l, int dado);  
void imprimelistacrescente(tlista l);  
void imprimelistadecrescente(tlista l);  
int listavazia(tlista l);
```

Para a lista duplamente encadeada, não é necessário usar duas variáveis para o percurso porque nessa versão, nós conseguimos saber quem é o anterior de um nó. Mas, vamos continuar implementando a operação de inclusão usando a variável auxiliar aux. O novo nó sempre vai ser inserido antes de aux.

// o novo nó é sempre inserido antes de aux

```
void crialista(tlista *l){  
    l->com = NULL; l->fim = NULL;  
}
```

```
int listavazia(tlista l){  
    if (l.com == NULL)  
        return 1;  
    else return 0;  
}  
//poderia ser implementada com essa linha somente com o código abaixo
```



MINISTÈRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

```
// return !(l.com);
}

int inserelista(tlista *l, int dado){
    tno *no, *aux;
    no = (tno *)malloc(sizeof(tno));
    if(!no) return 0;
    else {
        no->dado = dado;
        if (listavazia(*l)){ // lista vazia
            no->ant = NULL;
            no->prox = NULL;
            l->com = l->fim = no;
            return 1;
        }
        aux = l->com;
        while (aux && aux->dado < no->dado){
            aux = aux->prox;
        }
        if (aux){ // se o aux não for nulo significa que o novo nó será incluído antes de algum
            nó já existente
            if (aux->ant == NULL){
                /*nesse caso, o novo nó é o novo primeiro no da lista pois o anterior do aux é nulo
                (aux era o primeiro nó da lista)*/
                no->prox = l->com;
                no->ant = NULL;
                l->com->ant = no;
                l->com = no;
            }
            else {
                /*novo nó está entre dois nós da lista, logo nó anterior do aux deve ser atualizado
                também*/
                aux->ant->prox = no;
                no->ant = aux->ant;
                no->prox = aux;
                aux->ant = no;
            }
        }
        else {
            /*o novo nó é o novo último nó da lista
            l->fim->prox = no;
            no->ant = l->fim;
            l->fim = no;
        }
        return 1;
    }
}
```



MINISTÈRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

}

```
int removelista(tlista *l, int dado){
    tno *no, *aux;
    if (listavazia(*l)) return 0;
    aux = l->com;
    while (aux && aux->dado != dado){
        aux = aux->prox;
    }
    if (aux){ //encontrei o nó a ser removido
        if (!aux->ant){ //será removido o primeiro nó da lista
            if (aux->prox) { //se o no não for o único nó da lista
                aux->prox->ant = NULL;
            }
            else l->fim = NULL;
            l->com = aux->prox;
        }
        else { // o nó que será removido não é o primeiro
            aux->ant->prox = aux->prox; // o prox do anterior dele deve ser atualizado
            if (aux->prox) // o nó removido tem prox? se tiver, tem que atualizar o anterior do
prox dele
                aux->prox->ant = aux->ant;
            else // caso o nó removido não tenha aux, ele é o novo último nó
                l->fim = aux->ant;
        }
        free(aux);
        return 1;
    }
    else return -1;
}
```

```
void imprimelistacrescente(tlista l){
    tno *aux = l.com;
    printf("\nA lista e: \n");
    while (aux){
        printf("%d ", aux->dado);
        aux = aux->prox;
    }
}
```

```
void imprimelistadescrecente(tlista l){
    tno *aux = l.fim;
    printf("\nA lista e: \n");
    while (aux){
```



MINISTÈRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

```
printf("%d ", aux->dado);  
aux = aux->ant;  
}}
```